MOBILE APPLICATION DEVELOPMENT

BSC (HONS) IN INFORMATION TECHNOLOGY

Android Database Connectivity

Local Storage (SQLite / Room), Cloud Databases (Firebase) & Data Synchronization



Learning Outcomes

01 Explain why Android applications require persistent data storage.

02 Select an appropriate storage approach: SharedPreferences, SQLite, Room, or cloud storage.

03 Describe SQLite architecture and perform basic CRUD operations.

04 Explain Room’s Entity–DAO–Database architecture and its advantages over raw SQLite.

05 Describe Firebase as a cloud-backed data platform for Android applications.

06 Compare common data synchronization strategies and identify suitable approaches.

Why Is Data Persistence Required?

- Mobile applications must retain important data after the app is closed.
- Data should survive device restarts and configuration changes such as screen rotation.
- Typical requirements include remembering a logged-in user, storing registered users, and displaying a user list.

Without persistence

Data may disappear when the process or screen is recreated.

With persistence

Data can be restored when the application is opened again.

Design goal

Choose storage based on data size, structure, sharing, and connectivity.

Android Data Storage: Choosing the Right Layer

SharedPreferences

Small key–value data such as session flags and app settings.

SQLite

Structured relational data stored locally using SQL.

Room

Modern Android abstraction over SQLite with type-safe DAOs.

Cloud / Firebase

Remote data shared across devices and users; requires network-aware design.

- Key decision: Is the data only for this device, or must it be available across devices/users?
- For structured local data, Room is generally the preferred Android approach in modern applications.

Local vs Cloud Storage

Local Database

Stored on the device.

Fast local access.

Works offline.

Device-specific unless synchronized.

Cloud Database

Stored remotely.

Shared across devices.

Centralized data.

Network dependency and security rules.

Hybrid Approach

Local cache + cloud source.

Offline-first experience.

Synchronization required.

Handles conflicts and stale data.

What Is SQLite?

- SQLite is an embedded relational database.
- It uses SQL syntax and stores the database locally on the Android device.
- Android provides native SQLite support.

SharedPreferences limitation

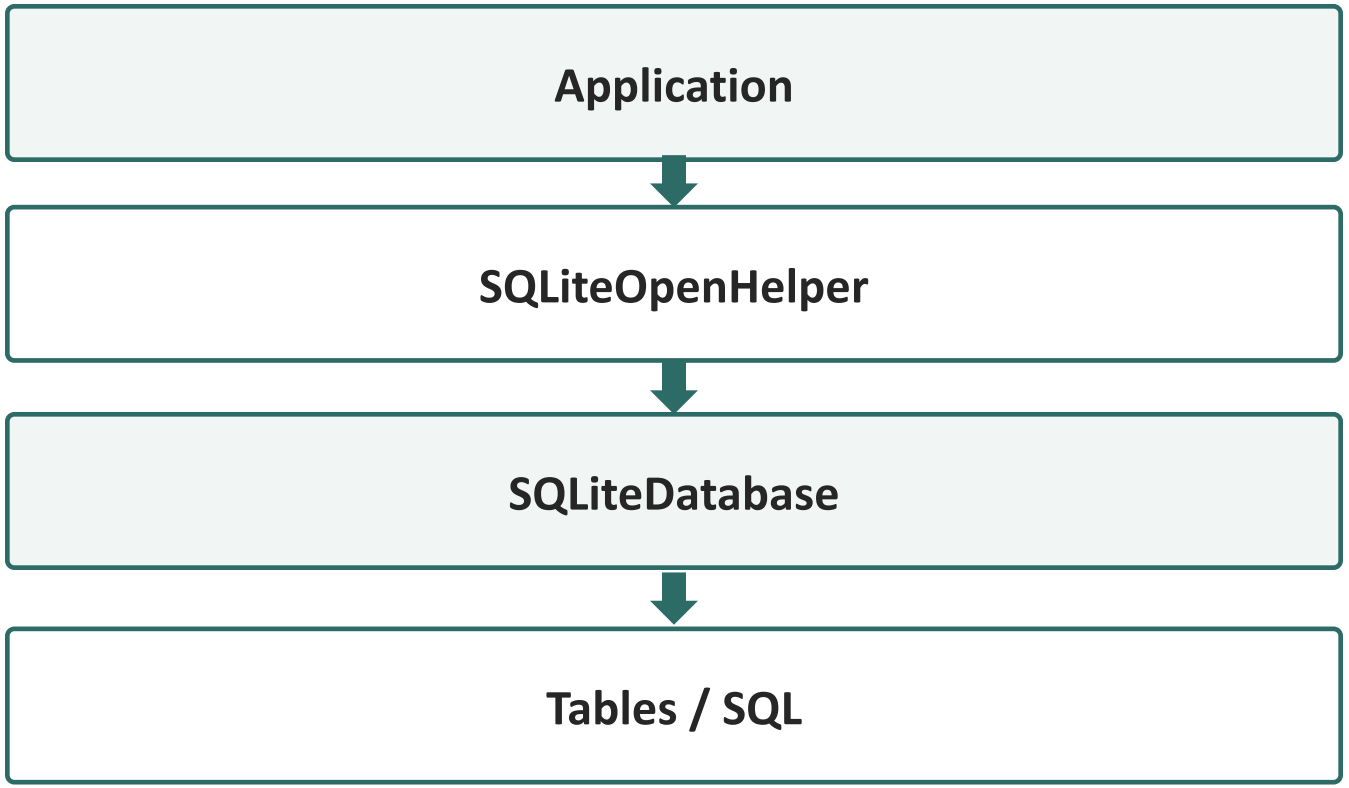
No table structure, relational model, or general SQL query capability.

SQLite advantage

Structured tables + SQL querying + local persistence.

SQLite is appropriate when an application needs tables, relationships, filtering, and SQL queries.

SQLite Architecture in Android



Manages database creation and version upgrades.

Exposes readable/writable access.

Executes SQL operations against tables.

Creating a SQLite Database Helper

```
class DBHelper(context: Context) :
    SQLiteOpenHelper(context, "UserDB", null, 1) {

    override fun onCreate(db: SQLiteDatabase) {
        val sql = """
            CREATE TABLE users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                username TEXT, email TEXT, password TEXT
            )
        """
        db.execSQL(sql)
    }

    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        db.execSQL("DROP TABLE IF EXISTS users")
        onCreate(db)
    }
}
```

SQLite CRUD Operations

CREATE / INSERT

Use ContentValues and db.insert(...).

READ / SELECT

Use query() or rawQuery() and process a Cursor.

UPDATE

Use ContentValues and db.update(...).

DELETE

Use db.delete(...) with a selection condition.

```
val values = ContentValues().apply {
    put("username", username)
    put("email", email)
    put("password", password)
}
db.insert("users", null, values)
```

Reading Data with SQLite

```
val cursor = db.rawQuery(
    "SELECT * FROM users WHERE email=? AND password=?",
    arrayOf(email, password)
)
if (cursor.moveToFirst()) {
    // Login success
}

val users = db.rawQuery("SELECT * FROM users", null)
while (users.moveToNext()) {
    val name = users.getString(users.getColumnIndexOrThrow("username"))
}
```

- Use parameterized selections (?) rather than building SQL strings with user input.
- Close Cursor/database resources appropriately when using lower-level SQLite APIs.

Problems with Raw SQLite

- Manual SQL queries create more boilerplate code.
- SQL mistakes may only become visible at runtime.
- Cursor handling and column mapping are repetitive.
- Database schema changes require careful migration/version management.
- Large codebases become harder to maintain and test.

Modern solution: Room provides an abstraction over SQLite while keeping the relational database underneath.

What Is Room?

- Room is part of Android Jetpack.
- Room is an abstraction layer over SQLite; it does not replace the underlying relational database.
- SQL queries are validated at compile time.

Entity

Represents a database table.

DAO

Defines insert, query, update, and delete operations.

Database

Holds the database and exposes DAOs.

Room Architecture: Entity → DAO → Database

Entity

Kotlin data class

@Table mapping

DAO

Database operations

@Insert / @Query / @Update / @Delete

Database

RoomDatabase

DAO access + configuration

Application / ViewModel calls the DAO rather than writing SQL throughout the UI layer.

Room Entity: Mapping a User Table

```
@Entity(tableName = "users")
data class User(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val username: String,
    val email: String,
    val password: String
)
```

@Entity

Marks the class as a Room table.

@PrimaryKey

Defines the unique identifier; autoGenerate creates IDs automatically.

Columns

Class properties become table columns unless configured otherwise.

Room DAO: Defining Database Operations

```
@Dao
interface UserDao {

    @Insert
    suspend fun insertUser(user: User)

    @Query("SELECT * FROM users")
    suspend fun getAllUsers(): List<User>

    @Query("""
        SELECT * FROM users
        WHERE email = :email AND password = :password
    """)
    suspend fun login(email: String, password: String): User?
}
```

Room Database Class & Instance

```
@Database(entities = [User::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}

val db = Room.databaseBuilder(
    applicationContext,
    AppDatabase::class.java,
    "app_database"
).build()
```

Database class

Lists entities and exposes DAO interfaces.

Database instance

Room’s builder creates the local database instance.

Versioning

Increase the version when the schema changes and define migrations as required.

CRUD Operations Using Room

Create

@Insert

`insertUser(user)`

Read

@Query

`getAllUsers()`

Update

@Update

`updateUser(user)`

Delete

@Delete

`deleteUser(user)`

```
lifecycleScope.launch {
    db.userDao().insertUser(User(username, email, password))
    val users = db.userDao().getAllUsers()
}
```

SQLite vs Room

Raw SQLite

Direct database APIs.

Manual SQL + Cursor handling.

More boilerplate.

Useful for understanding fundamentals.

Room

SQLite abstraction.

Entity + DAO + Database.

Compile-time query validation.

Cleaner and easier to maintain.

Key idea: Room still uses SQLite locally; it improves how the application interacts with it.

Why Use a Cloud Database?

- Local databases are tied to a particular device.
- A cloud database can provide a shared source of data for multiple users and devices.
- Cloud services can support real-time updates, centralized access, authentication integration, and scalable backend capabilities.
- Applications must account for network availability, latency, security rules, and synchronization.

Example: a student directory stored in the cloud can be accessed from a student's phone, tablet, or another authorized device.

Firebase: Cloud Data Options

Cloud Firestore

Document-oriented NoSQL database.

Collections → Documents → Fields

Supports queries, offline caching, and real-time listeners.

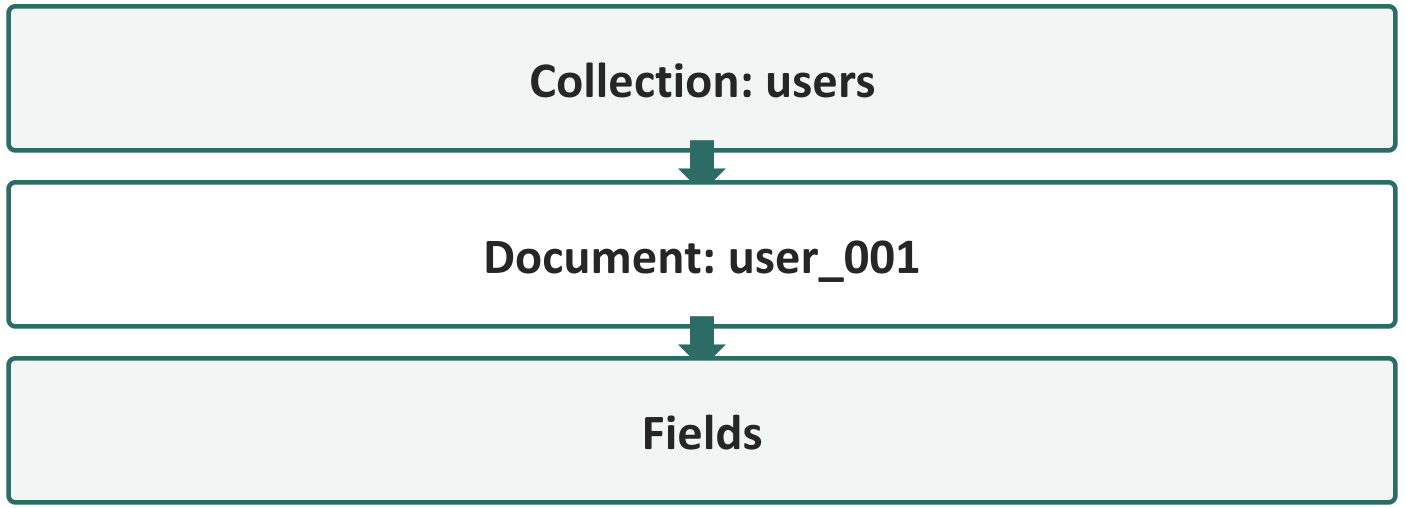
Realtime Database

JSON tree-based cloud database.

Useful for low-latency synchronized data and real-time state.

For a new application, choose the Firebase database based on the data model, query requirements, real-time behavior, and project constraints.

Firestore Data Model

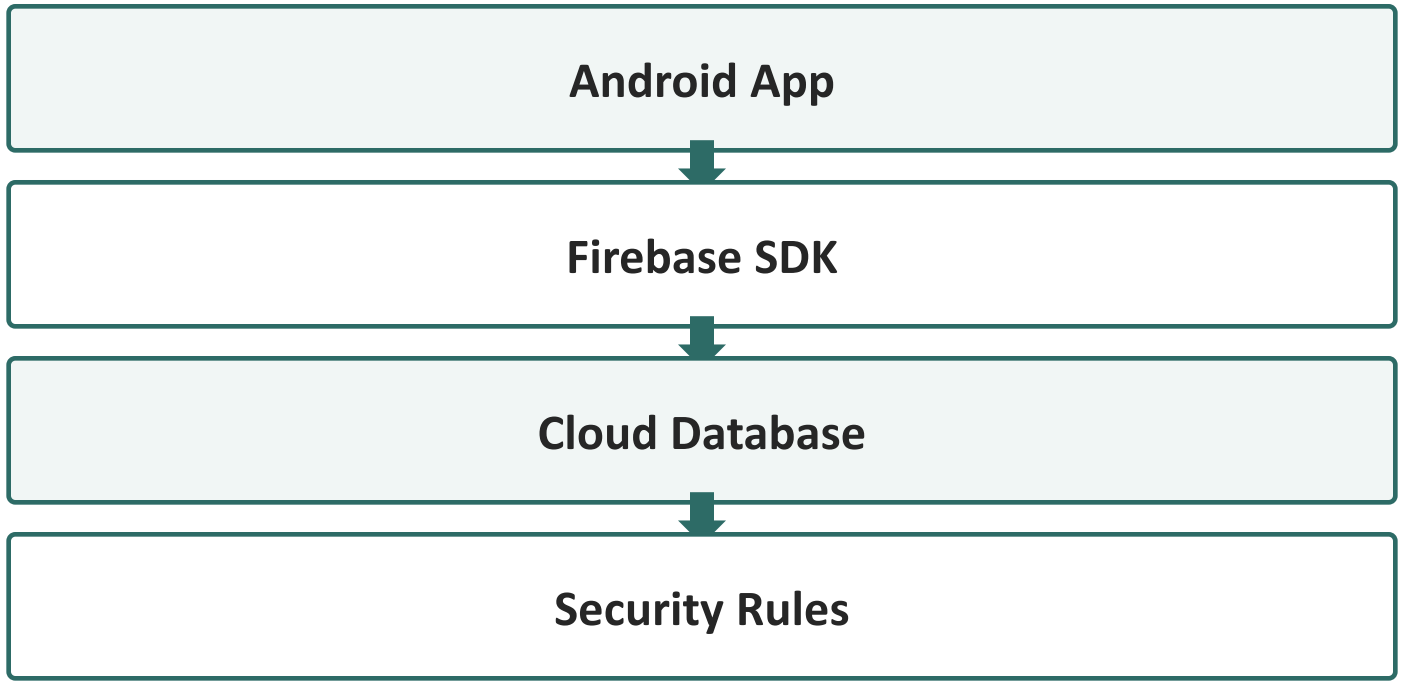


name
“Nimal”

email
“nimal@...”

department
“IT”

Firebase Android Flow



- The application communicates through Firebase SDKs rather than opening a direct database connection.
- Authentication and authorization rules determine who can access which data.

Cloud Database Example: Firestore Read

```
val db = FirebaseFirestore.getInstance()

db.collection("users")
    .get()
    .addOnSuccessListener { result ->
        for (document in result) {
            val name = document.getString("name")
            val email = document.getString("email")
            // Update UI / ViewModel with data
        }
    }
    .addOnFailureListener { exception ->
        // Handle network or permission error
    }
```

- Real applications should move data work out of the UI thread and expose loading/error/success states to the UI.

What Is Data Synchronization?

- Synchronization is the process of keeping data consistent between two or more locations.
- Typical Android example: local Room database ↔ cloud database.
- Synchronization must handle connectivity changes, retries, duplicate updates, stale data, and conflicting edits.
- The goal is not simply ‘copy everything’; it is to define clear rules for what data is authoritative and when updates are exchanged.

Source of truth: decide whether the cloud, local database, or a defined conflict-resolution rule is authoritative.

Common Synchronization Strategies

Online-only

Read/write directly against cloud. Simple architecture, but poor offline experience.

Periodic Sync

Upload/download at intervals or when the app starts. Lower continuous network use.

Event-driven / Real-time

Cloud changes trigger updates to connected clients. Useful for collaborative/live data.

Offline-first

Local database is used immediately; changes synchronize when connectivity returns.

Offline-First Synchronization Flow

01 User changes data

02 Save locally

03 Mark pending sync

04 Network available

05 Push changes

06 Pull remote changes

07 Resolve conflicts

08 Update local DB

Handling Synchronization Conflicts

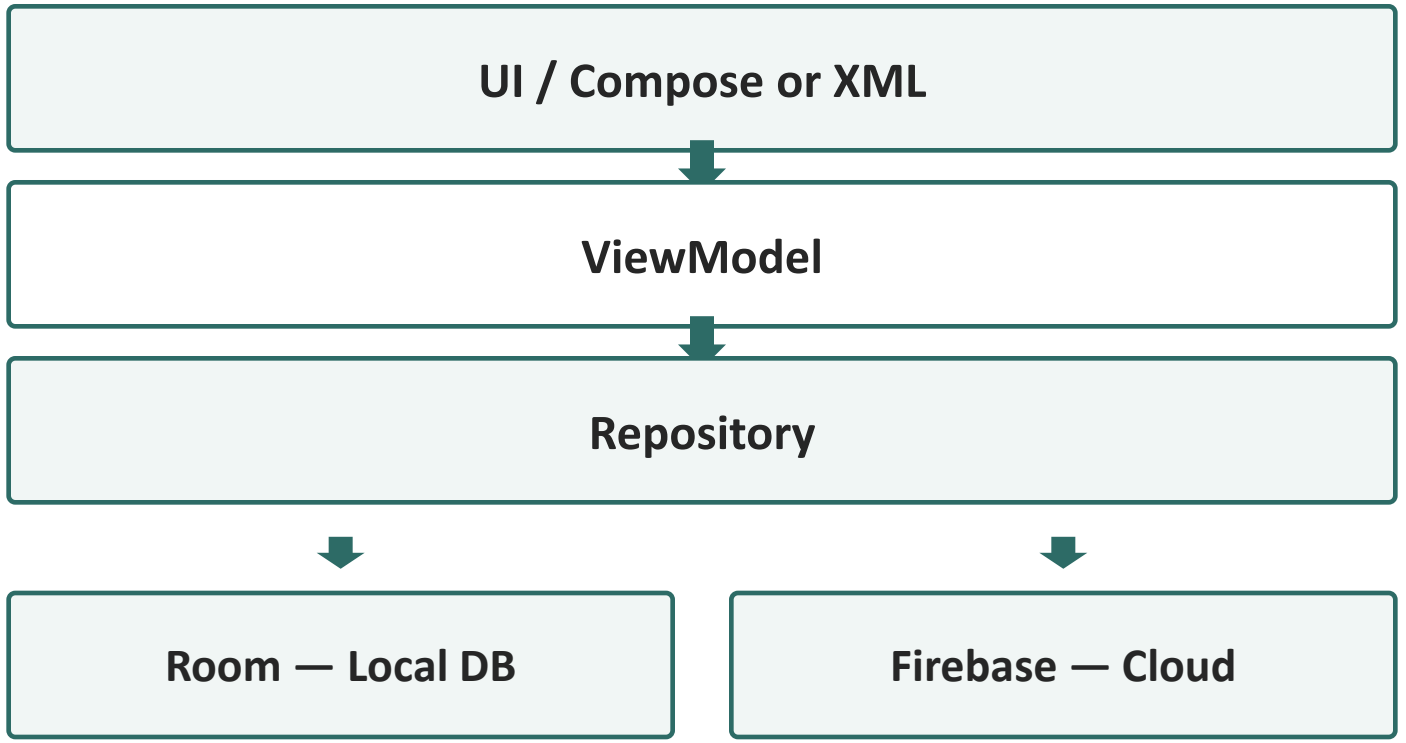
- Conflicts occur when the same logical record is changed in different places before synchronization completes.
- Last-write-wins is simple but can overwrite a meaningful change.
- Timestamp/version-based rules can help identify newer or concurrent updates.
- Field-level merging can preserve independent changes when the data model supports it.
- For critical data, conflict resolution should be an explicit business rule—not an accidental side effect.

Example: two devices edit the same student's phone number while offline. Which value should win when both devices reconnect?

Validation, Security & Reliability

- Validate data before saving or synchronizing it.
- Check required fields, email format, password rules, and duplicate records.
- Never trust client-side validation alone for sensitive or authoritative data.
- Use authenticated access and database security rules for cloud data.
- Handle loading, success, empty, offline, permission-denied, and failure states.
- Avoid storing plaintext passwords in production applications; use appropriate authentication mechanisms.

Recommended Architecture for a Modern Android App



The Repository can coordinate local and remote sources and expose a single data API to the ViewModel.

Decision Guide: Which Database Should You Use?

Small settings / flags	Structured local data	Shared remote data	Offline + cloud
SharedPreferences / appropriate key-value storage	Room over SQLite	Firebase Firestore / Realtime Database	Room local cache + Firebase + synchronization strategy

- Ask: What is the data structure? Must the data work offline?
- Ask: Must multiple devices/users see the same data? How frequently does the data change?
- Ask: What happens when two changes conflict?

Mini Case Study: Student Directory

- Requirement: display at least 10 students with name, registration number, and department.
- Local requirement: the directory should remain viewable without internet.
- Cloud requirement: authorized staff can add or update student records.
- Suggested design: Room stores the local copy; Firebase stores the shared cloud copy.
- Synchronization: fetch remote changes when connected, update Room, and display the Room data in RecyclerView.

Student task: sketch the data flow and identify where validation, authentication, caching, and synchronization occur.

Key Takeaways

01 Persistence allows Android applications to retain data beyond the current screen/process lifetime.

02 SQLite provides local relational storage and SQL operations.

03 Room provides a cleaner, type-safe abstraction over SQLite using Entity, DAO, and Database components.

04 Firebase provides cloud-backed storage for applications that need shared remote data.

05 Real-world applications often combine Room + cloud storage to support offline use and synchronization.

06 Synchronization requires explicit rules for connectivity, retries, ordering, and conflicts.

Discussion / Quick Check

01 Why is SharedPreferences unsuitable for a large list of students?

02 What is the role of SQLiteOpenHelper?

03 How are Entity, DAO, and Database related in Room?

04 Why is Room generally preferred over raw SQLite for modern Android applications?

05 What is the difference between local and cloud data storage?

06 Why can synchronization create conflicts?

07 Give one situation where offline-first design is valuable.

Thank You

Questions & Discussion